6COSC022C 2

Advanced Server-Side Web Programming

Who is teaching on this module?

- Module team: Indrajith Ekanayake (lectures), Mr Vathila De Silva (Tutorials), Ms Tharushi Pramoda (Tutorials)
- Email: indrajith.e@iit.ac.lk, vathila.s@iit.ac.lk, tharushi.p@iit.ac.lk
- Email us for any module-related matters. CC all the module team

Module aims

- Look at examples of the technologies used in building big modern web applications
  - Get some experience in using them
  - Also experience in using simple MVC frameworks
- Typical workflows –> e.g., user authentication
- Security issues
- Experience building applications using MVC

Content

A rough outline of the topics we will cover on this module, and their order is:

- Web frameworks for php
- Ajax and jQuery
- XML and JSON for client/server data exchange
- RESTful APIs
- MVC frameworks for JavaScript - Backbone
- User authentication (this may come earlier)
- General topics – security, scalability etc.

Assessment

- Two courseworks
  - First coursework – design of web application
  - Second coursework – implementation of your application
- CW1 = 40% and CW2 = 60%

Resources

- See Blackboard module website for full details
- You will need:
  - Access to a machine (Mac or Windows laptop/PC)
  - Text editor
  - Web server, PHP, MySQL
    - Get full stack bundle – MAMP (for Macs), XAMPP (for Windows or Mac)
  - Web browsers
    - At least Chrome AND Firefox

Today

- Web application overview
  - What makes them special?
- Basics of web infrastructure
  - URLs
  - Request methods
- Using web browser developer tools

What is this module about?

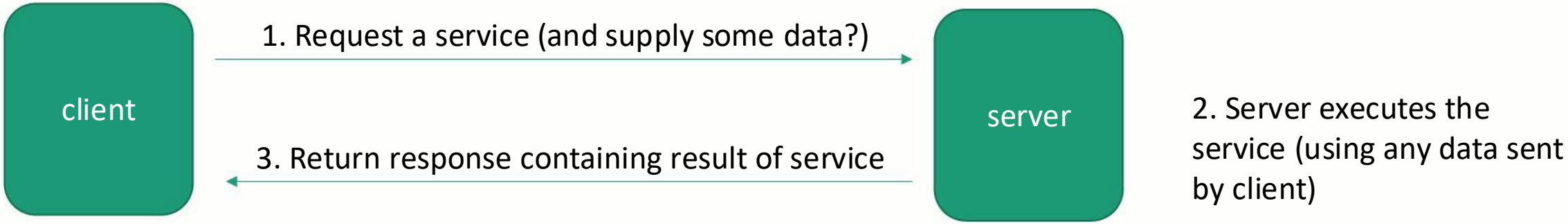
- This module is about writing web applications!
- Web applications are a special kind of application
- What makes them special?
  - They run on the web
- Running on the web has consequences…
  - … for the organization and structure of the application
  - … for what we (as developers) are able to do with the application
  - Etc.
- The most important consequence is in terms of complexity

Web applications

- Web applications are client-server applications – a class of distributed applications
- One part of the web application runs on one machine (the ”client”), the other part runs on another machine (the “server”)
- Both machines are connected to a network, so they can communicate
- The “client” part of the application runs in a web browser
- The “server” part of the application is run by a web server
- Communication between the two is via HTTP (Hypertext Transfer Protocol) messages passed between the client and the server

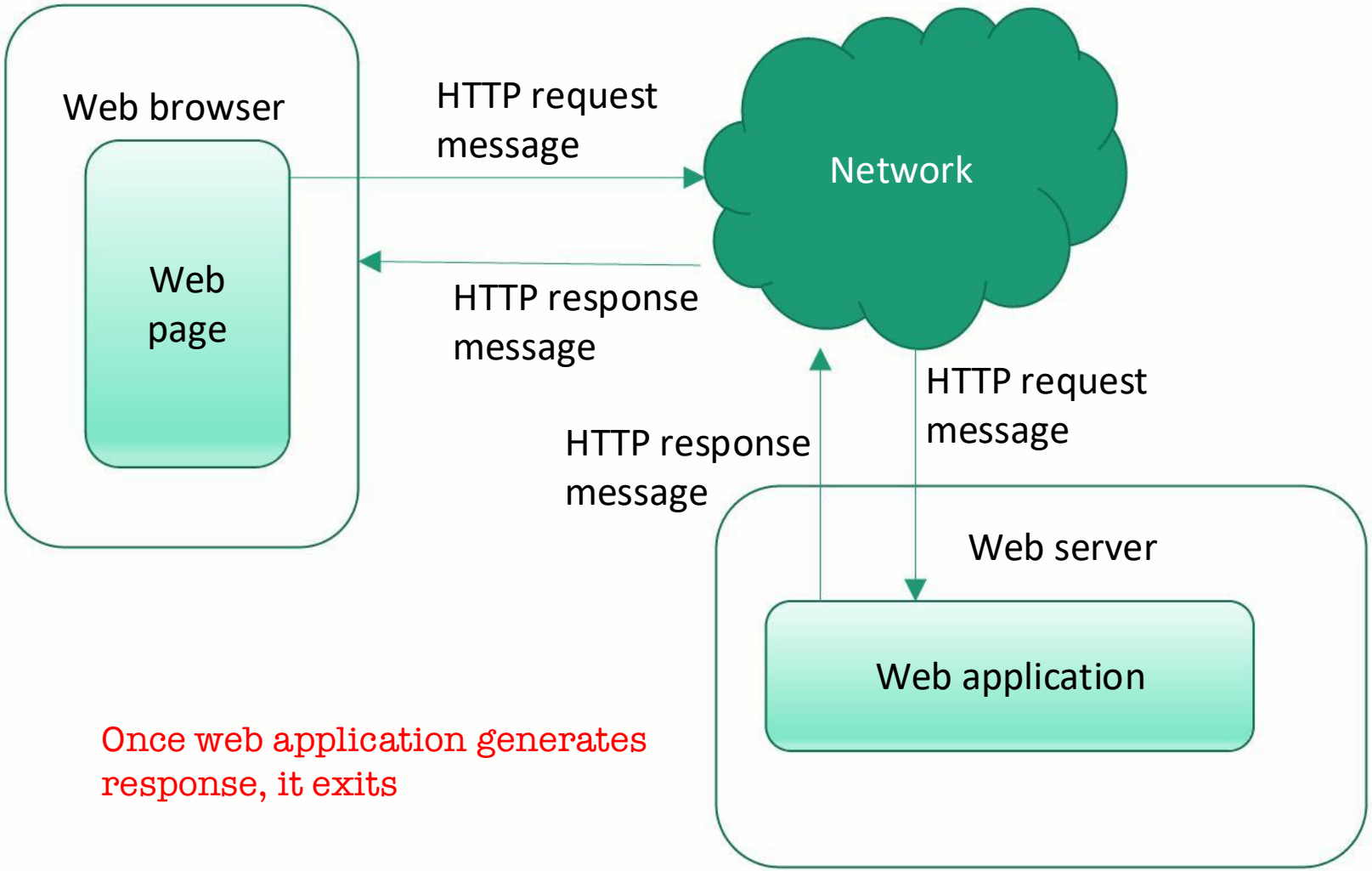
Client-Server

- General paradigm for distributed applications – including web applications



- Used a lot on applications that use a network/internet
- Application functionality divided across client and server, which run on (potentially) different platforms in different locations
- Other examples: FTP, chat servers, mobile banking apps etc.

Web application architecture



Once web application generates response, it exits

The web page and the web application code make one application, divided across different platforms and locations – hence “distributed”.

The different parts of the application communicate via message passing across a network

Once the web application exits, any data held in program variables (e.g, function variables) will be lost – so how do web applications remember user data? They can’t – hence the need for cookies and databases

Web application example

snapchat - Google Search

www.google.co.uk/search

snapchat

All Images News Videos Shopping More Settings Tools

About 475,000,000 results (0.57 seconds)

Snapchat - The fastest way to share a moment!
https://www.snapchat.com/
Snapchat lets you easily talk with friends, view Live Stories from around the world, and explore news in Discover. Life's more fun when you live in the moment!
Download · Snapchat Support · What is Snapchat? · Snapchat Ads

Snapchat (@Snapchat) · Twitter
https://twitter.com/Snapchat
www.snap.com/en-US/news...
2 days ago · Twitter

Snapchat – Apps
https://play.google.com
Life's more fun when you
always capture or save
You visited this page on

1. Click this link

```
Host: www.snapchat.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.12; rv:62.0)
Gecko/20100101 Firefox/62.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://www.snapchat.com/
Cookie: country=GB
Connection: keep-alive
Upgrade-Insecure-Requests: 1
TE: Trailers
```

2. Send this HTTP request

Snapchat web server + application

Snapchat – the fastest way to

www.snapchat.com/l/en-gb/

What is Snapchat?

INTRODUCING

Snap Kit

A simple and secure way for developers to integrate with Snapchat's greatest features.

Learn more

3. Server sends back response to browser containing HTML code to display this page

Implications of being distributed

- If an application is distributed…
  - Then different parts will be in different places on the network

- This has two main implications
  1. **Structure**
     - How do the different parts know where the other parts are? How do they identify and locate them?
  2. **Communication**
     - How do they communicate? What do they use to communicate?

Structure and Identity - URLs

- URLs are the way we uniquely identify and address resources on the web
- A web application may be made up of many URLs – each one identifying one part of a whole web application
- E.g.

  http://www.example.com/shop/products/browse.php?category=laptops&maxprice=500

- A URL helps a browser identify
  - Which server it should communicate with? (example.com)
  - What resource it should ask the server for? (/shop/products/browse.php?category=laptops&maxprice=500)
  - What message protocol should it use to send messages to server? (http)

URLs

- A URL has many parts:

http://www.example.com/shop/products/browse.php?category=laptops&maxprice=500

  - Protocol = “http”
  - TLD (top-level domain) = “.com”
  - Domain = “example.com”
  - Sub-domain = “www” (within example.com domain)
  - Path = “/shop/products/browse.php” – identifies path to resource from web document root folder
  - Query string = “category=laptops&maxprice=500” – name=value pairs separated by ‘&’

URLs and Resources

- A URL is an address of a resource
- E.g, `http://www.example.com/shop/products/browse.php?category=laptops&maxprice=500`
  - Defines a resource managed by the web server at www.example.com
  - In this case, the URL refers to a resource that is a file under the server' document root – the server identifies it as a PHP file based on file extension and runs it through the PHP interpreter to get the program output, rather than just sending the file contents back to the brower
- Not all URLs address "physical" resources
- Some URLs address "logical" resources, e.g.,
  - `https://www.amazon.co.uk/HP-Pavilion-13-an0006na-13-3-Inch-Laptop/dp/B07KRWHX9Q/`

Domains

- What is a domain name?
- A name that identifies part of a network or a machine on a network
- Used to make it easy to remember and refer to some resource on a network
  - A network-connected printer
  - A machine running a server (web server, FTP server etc.)
- Domain name only used for humans – needs translating to IP address for machine-machine communication
  - DNS (Domain Name Service) – handles Domain Name <-> IP Address translation

What happens when you type a URL?

1. You type facebook.com into your browser

2. Your web browser sends request to DNS to translate “facebook.com” to IP address

3. DNS translates “facebook.com” to IP address (e.g., 31.13.70.36) and sends to browser in response

4. Browser sends HTTP request to server listening on machine 31.13.70.36

Communication - HTTP

- The browser and web server communicate using HTTP (Hypertext Transfer Protocol)
- HTTP lays out rules and formats for request and response messages
  - Browser (client) – request messages
  - Web server (server) – response messages

- Web applications don’t need to know about specifics of dealing with HTTP – this is dealt with by the browser and web server – but as developers, we need to know limitations etc.

Sending data in HTTP requests

- One of the key things we need to know about HTTP is how it transfers data from browser to server
- If we fill in a form and press submit, how is the form data sent to the server?
- Depends on request method – usually GET or POST
- If GET, then build query string and add to URL
- If POST, then add to end of HTTP request message

GET request method

- E.g. used in a form to send data

```html
<html>
<head>
    <title>Request method test</title>
</head>
<body>
<h2>Fill in the form</h2>

<form action="hello.php" method="GET">
    Name: <input type=text name="MYNAME">
    Age: <input type=text name='MYAGE'>
    <input type=submit value="Say hello">
</form>

</body>
</html>
```

- HTTP request will be
  - http://localhost:8888/formtest/hello.php?MYNAME=Simon&MYAGE=21
  - i.e., form data added to URL as query string – note how field names are used

POST request method

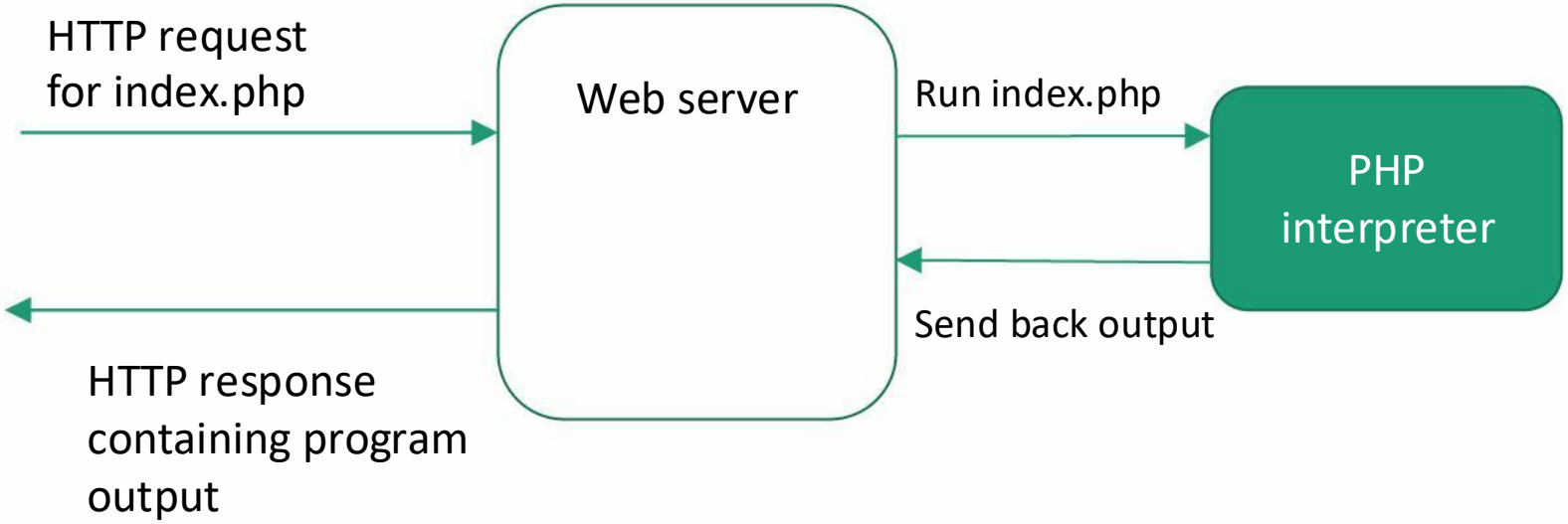
```
<form action="hello.php" method="POST">
    Name: <input type=text name="MYNAME">
    Age: <input type=text name='MYAGE'>
    <input type=submit value="Say hello">
</form>
```

- Like GET, but form data not added to URL in HTTP request message
- URL requested is
  - http://localhost:8888/formtest/hello.php
  - I.e, form data NOT added to URL
- Instead, form data is added at end of message (in separate section of message)
- Still formatted as
  - MYNAME=Simon&MYAGE=21

What happens on the server when HTTP request received?

- Server identifies what kind of resource has been requested?
- Static resource? HTML file, CSS file, image/video file etc.
  - Just send back file contents
  - With appropriate content-type header (text/html, application/json, image/jpeg)
- Dynamic resource? PHP/Java/Python etc.
  - Program to be executed
  - Output from program becomes content to send to browser

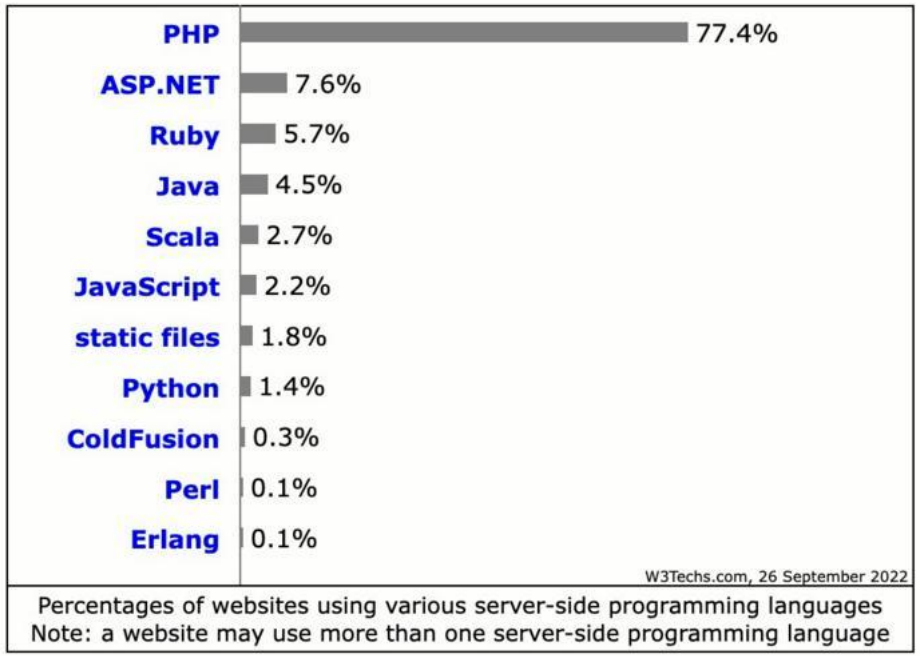
Executing programs on server



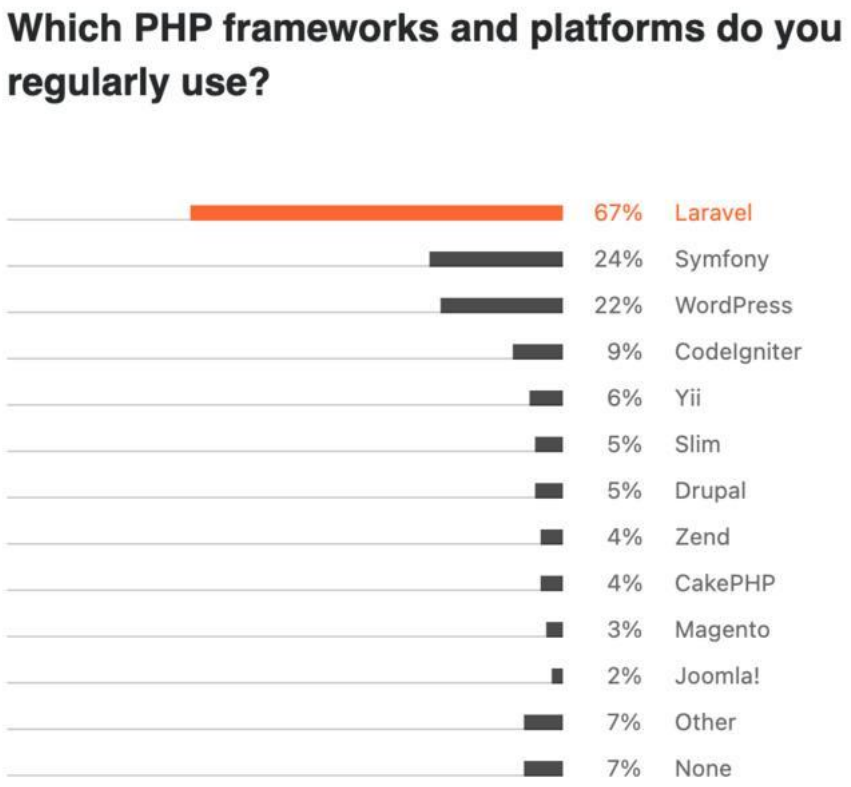
1. Web server looks at file extension
2. Sees it's a file with .php extension
3. Starts up PHP interpreter to run file
4. Intercepts printed output from running index.php
5. Sends back printed output in HTTP response

What is used to make a web application

- Server-side technologies – mostly:
  - Programming languages
  - Data repositories – databases such as MySQL, PostgresSQL etc.
  - Web Frameworks



From w3techs.com



Jetbrains survey 2021 : https://www.jetbrains.com/lp/devecosystem-2021/php/

What makes a language good for web programming?

- You can use almost any programming language for server-side web programming
- You just need a way to interface it with a web server – this is (mostly) just a matter of configuration
- But some languages are more popular for web development than others – is their popularity based on something that makes them better for this purpose?
- Suggestion – being better for web development is a matter of what features a language offers…

Web Frameworks

- Most interesting web applications are large
  - In terms of lines of code on server and on client
    - Facebook home page source code (the text of the .html page) is 1.6MB
  - Number of programs
    - e.g., browse, search, view shopping basket, review product, register, login etc. etc.
  - In terms of amount of data and number of different data items (e.g., tables and table columns)

- Remember – one of the most important consequences of an application running on the web is complexity
- We need a way to structure such applications to reduce/manage complexity => frameworks

CodeIgniter

- “Vintage” PHP web framework – has been around a long time
- Not as fully featured or modern as most PHP web frameworks, but
  - Excellent documentation
  - Easy to install
  - Learning curve not as steep as for other frameworks

- Other options – Symfony, Laravel, Zend etc.
  - Also require more up to date versions of PHP that we have installed!!!

- Frameworks, however, are object-oriented…

Summary and next week

- Web applications are an example of client-server distributed application
- Client-side and server-side are parts of the same application
  - Work together
  - URLs used by client-side to address server-side part of application
  - Message passing using HTTP protocol
- Tutorials – start next week
- Next week – OO PHP programming, introduction to MVC frameworks and CodeIgniter for PHP